Module 9

Option Pricing – Numerical Techniques

October 27 & 29, 2025

Risk neutral process for a stock S paying continuous dividend yield:

In the risk neutral (RN) world the expected rate of total return in S equals to r, the risk free rate. Let q be the continuous dividend yield,

$$dS = (r-q)Sdt + \sigma SdW_t$$

The price drift is $r-q$ so that after reinvest dividend at rate q the total return rate in S is r.

Thus in the RN world (see Module 7 for a stock process with drift μ, $\mu \to r-q$):

$$S_t = S_0 e^{\left(r-q-\frac{\sigma^2}{2}\right)t+\sigma W_t}$$

$$P(S_t \le Y) = \int_0^Y \frac{1}{\sigma x\sqrt{2\pi t}} e^{-\frac{\left(\ln x - \ln S_0 - \left(r-q-\frac{\sigma^2}{2}\right)t\right)^2}{2\sigma^2 t}} dx$$

$< S_t >= S_0 e^{(r-q)t}$, $< S_t^{\,2} > - < S_t >^2 = S_0^2 e^{2(r-q)t}(e^{\sigma^2 t} - 1)$

Let f be a European derivative on S with maturity T:

$$f(S,t) = e^{-r(T-t)} E_t\left[f(S_T, T)\right]$$

$E_t[.]$: RN expectation at time t

Binomial Tree Revisited

Parameters for binomial Trees:

Matching expectation and variance on a binomial tree:

$t \to t+\delta t$, $S_t \to S_t u, S_t d$ $(u > 1, 0 < d < 1)$ with RN probability $p_u = p$, $p_d = 1-p$:

$$E_t\left[S_{t+\delta t}\right] = pS_t u + (1-p)S_t d = S_t e^{(r-q)\delta t}$$

$$E_t\left[S_{t+\delta t}^2\right] - \left(E_t\left[S_{t+\delta t}\right]\right)^2 = pS_t^2 u^2 + (1-p)S_t^2 d^2 - S_t^2 e^{2(r-q)\delta t} = S_t^2 e^{2(r-q)\delta t}(e^{\sigma^2 \delta t} - 1)$$

In the Cox-Ross-Rubinstein (CRR, see Module 6) construction $d = 1/u$, to the leading order of δt:

$$p = \frac{e^{(r-q)\delta t} - d}{u - d}$$

$$u = e^{\sigma\sqrt{\delta t}}$$

$$d = e^{-\sigma\sqrt{\delta t}}$$

Cook book:

- Choose the number of time steps N and set tree building parameters:

  $$\delta t = \frac{T}{N}\ ,\ p = \frac{e^{(r-q)\delta t} - d}{u - d}\ ,\ u = e^{\sigma\sqrt{\delta t}}\ ,\ d = 1/u$$

- Grow the N-step tree for S:

  $$0 = t_0 \to t_1 = t_0 + \delta t \to t_2 = t_1 + \delta t \to \ldots \to t_N = T$$

  $$S \to \left(uS, dS\right) \to \left(u^2 S, udS, d^2 S\right) \to \left(u^3 S, u^2 dS, ud^2 S, d^3 S\right) \to \ldots$$

  Assume the final branch is $S_0^{t_N} < S_1^{t_N} < \ldots < S_N^{t_N}$

- Calculate the payoff f at the terminal branch $t_N = T$. For instance for a call

  $$f_i^{t_N} = \max(S_i^{t_N} - K, 0)$$

- Backward induction:

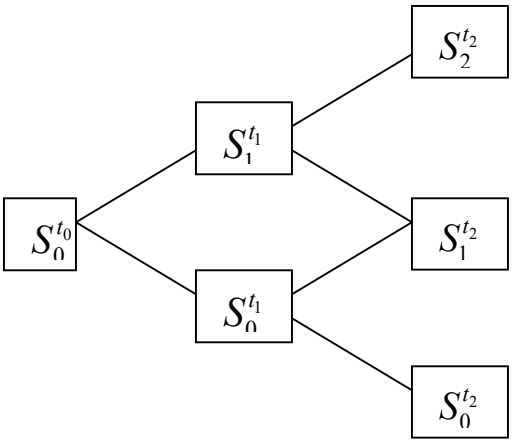
  $$f_i^{t_j} = e^{-r\delta t}\left[pf_{i+1}^{t_{j+1}} + (1-p)f_i^{t_{j+1}}\right]$$

  Check possible early exercise for American options

  → $f_0^{t_0}$ is the option price

Calculation of Greeks:

- Delta, gamma & theta: may be computed on the tree for better numerical stability



  - Delta: $\Delta \approx \frac{f_1^{t_1} - f_0^{t_1}}{S_1^{t_1} - S_0^{t_1}}$
  - Gamma: $\Gamma \approx \frac{\Delta_2 - \Delta_1}{S_1^{t_1} - S_0^{t_1}}$, where $\Delta_2 = \frac{f_2^{t_2} - f_1^{t_2}}{S_2^{t_2} - S_1^{t_2}}, \Delta_1 = \frac{f_1^{t_2} - f_0^{t_2}}{S_1^{t_2} - S_0^{t_2}}$
  - Theta: $\Theta \approx \frac{f_1^{t_2} - f_0^{t_0}}{2\delta t}$

  Works well in general for reasonably small δt

- Other greeks: standard finite difference:
  - Vega: $vega \approx \frac{f(\sigma + \delta\sigma) - f(\sigma)}{\delta\sigma}$
  - Rho: $\rho \approx \frac{f(r + \delta r) - f(r)}{\delta r}$

Practical issues:

- Interest rate term structure, dividend yield curve
- Volatility surface (skew and term structures)

  → trees with "local volatility" (Derman & Kani), …

- Default (S *jumps* to 0)
- Special events (e.g., discrete dividend, …)
- Special stock prices (e.g., barriers, …)
- Numerical convergence and stability

Monte Carlo Simulation

Evaluate integral using Monte Carlo method:

$I = \int_0^1 g(x)dx$

Let X be uniformly distributed random variable over [0, 1], then

$I = E[g(X)]$

Let $X_1, X_2, ..., X_n$ independently sampled from [0, 1]

→ $g(X_1), g(X_2), ..., g(X_n)$ iid with mean = I

Strong law of large number → $\lim_{n\to\infty} \frac{1}{n} \sum_{i=1}^{n} g(X_i) = E[g(X)] = I$

Where do we get the random variables?

- Quantum random number generators
- (pseudo) random number generators: numbers generated by algos, completely deterministic but if the algo is not known one may think they are random, then they may be good enough for one's purpose

Example:

Estimation of Pi:

```
import numpy as np

n = 10000000

x = np.random.uniform(low = 0, high = 1, size = n)

y = np.random.uniform(low = 0, high = 1, size = n)

radius = np.sqrt(x**2 + y**2)
```

```
circle = radius[radius <= 1]

pi_estimated = len(circle)*4.0/n

print('pi_estimated =', pi_estimated)

print('pi_exact =', np.pi)
```

Output:

```
pi_estimated = 3.1415912

pi_exact = 3.141592653589793
```

Error estimation:

Let

$$G_n = \frac{1}{n}\sum_{i=1}^{n} g(X_i)$$

$X_1, X_2, ..., X_n$ are independently sampled → variance of sum = sum of variance

$$Var(G_n) = \frac{1}{n^2}\sum_{i=1}^{n} Var(g(X_i)) = \frac{Var(g)}{n} \equiv \frac{\sigma_g^2}{n} \approx \frac{\sigma_n^2}{n}$$

where

$$\sigma_n^2 = \frac{1}{n-1}\sum_{i=1}^{n}\left(g(X_i) - G_n\right)^2 \quad \text{(unbiased estimator of } \sigma_g^2\text{)}$$

→ $\delta G_n \approx \frac{\sigma_n}{\sqrt{n}}$ as an error estimation

Confidence interval:

Assume $\{g_i\}$ has mean μ_g and standard deviation σ_g, central limit theorem:

$$\frac{g_1 + g_2 + ... + g_n - n\mu_g}{\sigma_g\sqrt{n}} = \frac{\sqrt{n}\left(G_n - \mu_g\right)}{\sigma_g} \sim N(0,1)$$

$1-\alpha$ confidence interval = $G_n \pm z_{\alpha/2}\frac{\sigma_g}{\sqrt{n}}$, where $N(-z_{\alpha/2}) = \alpha/2$

Since σ_g may not be known, one can use σ_n instead,

$1-\alpha$ confidence interval $\approx G_n \pm z_{\alpha/2}\frac{\sigma_n}{\sqrt{n}}$

95% confidence → $\alpha = 0.05$ → $z_{0.025} \approx 1.96$

Evaluate derivatives using Monte Carlo method:

Risk neutral pricing theory provided a natural framework for Monte Carlo simulations.

Let f be a European derivative product

$$f(S,t) = e^{-r(T-t)}E_t\left[f(S_T,T)\right]$$

1) Sample a random path for S in a risk-neutral world.
2) Calculate the payoff for the derivative.
3) Repeat steps 1) and 2) to get many sample values of the payoff from the derivative in a risk-neutral world.
4) Calculate the mean of the sample payoffs to get an estimate of the expected payoff in a risk-neutral world.
5) Discount this expected payoff at the risk-free rate to get an estimate of the value of the derivative.

How to generate paths for S?

- Differential form:

  $\delta S = (r-q)S\delta t + \sigma S\delta W$, $\delta W_t = \varepsilon\sqrt{\delta t}$, $\varepsilon \sim N(0,1)$

- Finite time form:

  $$S_{t+\delta t} = S_t e^{\left(r-q-\frac{\sigma^2}{2}\right)\delta t+\sigma\delta W_t}$$

  works better in general as δt is finite in actual simulations

- Entire path: sample the terminal distribution directly (if r, q, σ are constant)

  $$S_T = S_0 e^{\left(r-q-\frac{\sigma^2}{2}\right)T+\sigma W_T}$$

  fast but inflexible, hard to generalize

Example:

European call/put options:

```
import numpy as np


def option_payoff(S, K, option_type):
    payoff = 0.0
    if option_type == "call":
        payoff = max(S - K, 0)
    elif option_type == "put":
        payoff = max(K - S, 0)
    return payoff


S0 = 100
K = 100
T = 1.0
r = 0.045
q = 0.03
sigma = 0.35


option_type = "call"
n_steps = 100           # number of steps
np.random.seed(110124)  # make result reproducible
n_simulation = 10000    # number of simulation


dt = T/n_steps
sqrt_dt = np.sqrt(dt)
payoff = np.zeros((n_simulation), dtype = float)
step = range(0, int(n_steps), 1)
for i in range(0, n_simulation):
    ST = S0
```

```
    for j in step:
        epsilon = np.random.normal()
        ST *= np.exp((r - q - 0.5*sigma*sigma)*dt +
                     sigma*epsilon*sqrt_dt)
    payoff[i] = option_payoff(ST, K, option_type)

option_price = np.mean(payoff)*np.exp(-r*T)
print(option_type + ' price =', round(option_price, 4))
```

Output:

`call price = 14.0225` (BSM: `14.1157`)

Increase the number of simulated paths:

`n_simulation = 100000` → `call price = 14.1453`

`n_simulation = 1000000` → `call price = 14.0991`

Error estimation → homework

Variance reduction methods:

- Importance sampling:

  Consider

  $$I = \int_a^b g(x)dx$$

  If $g(x)$ is significantly different from zero only in a sub-region of [a, b] then uniformly sampling [a, b] may not be efficient. Consider a function $h(x)$ that is also predominant in the same sub-region as $g(x)$ and

  $$h(x) \geq 0,\ \int_a^b h(x)dx = 1$$

  $$I = \int_a^b g(x)dx = \int_a^b \frac{g(x)}{h(x)} h(x)dx = E_h\left[\frac{g(x)}{h(x)}\right]$$ → sampling according to $h(x)$

  Especially useful if one can sample from the terminal distribution directly

- Antithetic variate:

Consider

$$I_g^{2n} = \frac{1}{2n}\sum_{i=1}^{2n} g_i$$

and

$$I_{gf}^{n} = \frac{1}{2n}\sum_{i=1}^{n} \left(g_i + f_i\right),$$

where f_i has the same mean and variance as g_i, $\{(g_i, f_i)\}$ are independent but g_i and f_i are dependent with correlation coefficient ρ_{gf}

→ $\lim_{n\to\infty} I_g^{2n} = \lim_{n\to\infty} I_{gf}^{n} = I$

$$Var(I_g^{2n}) = \frac{\sigma_g^2}{2n},$$

$$Var(I_{gf}^{n}) = \frac{1}{4n^2}\sum_{i=1}^{n} Var\left(g_i + f_i\right)$$

$$Var\left(g_i + f_i\right) = Var\left(g_i\right) + Var\left(f_i\right) + 2Cov\left(g_i, f_i\right) = \sigma_g^2 + \sigma_f^2 + 2\rho_{gf}\sigma_g\sigma_f = 2\sigma_g^2 + 2\rho_{gf}\sigma_g^2$$

→ $Var(I_{gf}^{n}) = \frac{\sigma_g^2}{2n} + \frac{\rho_{gf}\sigma_g^2}{2n} = \frac{\sigma_g^2}{2n}\left(1 + \rho_{gf}\right)$

→ variance is reduced if ρ_{gf} is negative

In practice ρ_{gf} is often not known or difficult to compute thus when estimating error one may consider

$$x_i = \frac{g_i + f_i}{2}$$

$\{x_i\}$ are independent → $Var(I_{gf}^{n}) = \frac{\sigma_x^2}{n}$

Example:

Consider a pair of paths

$$S_t = S_0 e^{\left(r - q - \frac{\sigma^2}{2}\right)t + \sigma W_t}, \quad S_t' = S_0 e^{\left(r - q - \frac{\sigma^2}{2}\right)t - \sigma W_t}$$

→ $\rho_{S_t S'_t} = \dfrac{e^{-\sigma^2 t} - 1}{e^{\sigma^2 t} - 1} < 0$

Needs to look at case by case for specific option payoff f but often $\rho_{f_S f_{S'}} < 0$

→ starting with S_0, sample pair of paths

$$S_{t+\delta t} = S_t e^{\left(r - q - \frac{\sigma^2}{2}\right)\delta t + \sigma \delta W_t}, \quad S'_{t+\delta t} = S'_t e^{\left(r - q - \frac{\sigma^2}{2}\right)\delta t - \sigma \delta W_t}$$

- Control variate:

  Consider again

  $$I_g^n = \frac{1}{n}\sum_{i=1}^{n} g_i$$

  and

  $$I_h^n = \frac{1}{n}\sum_{i=1}^{n} h_i \equiv \frac{1}{n}\sum_{i=1}^{n}\left(g_i - \beta(f_i - \mu_f)\right)$$

  $$I_h^n = \frac{1}{n}\sum_{i=1}^{n} g_i - \beta\left(\frac{1}{n}\sum_{i=1}^{n} f_i - \mu_f\right) = I_g^n - \beta(I_f^n - \mu_f)$$

  where $\{f_i\}$ are iid with $E(f) = \mu_f$. g_i and f_i, not necessarily have the same distribution, have correlation ρ_{gf}. However $\{h_i\}$ are independent and identically distributed

  → $\lim_{n\to\infty} I_g^n = \lim_{n\to\infty} I_h^n = I$

  $$Var(I_g^n) = \frac{\sigma_g^2}{n}$$

  $$Var(I_h^n) = \frac{1}{n^2}\sum_{i=1}^{n} Var(h_i) = \frac{1}{n}\left(\sigma_g^2 - 2\beta\rho_{gf}\sigma_g\sigma_f + \beta^2\sigma_f^2\right)$$

  Choose β to minimize variance:

  $$\beta^* = \rho_{gf}\frac{\sigma_g}{\sigma_f} \quad \rightarrow \quad Var(I_h^n) = \frac{\sigma_g^2}{n}\left(1 - \rho_{gf}^2\right)$$

In practice ρ_{gf} is often not known hence β^* needs be estimated:

$$\beta^* = \rho_{gf}\frac{\sigma_g}{\sigma_f} = \frac{Cov(g,f)}{\sigma_f^2} \;\rightarrow\; \hat{\beta}^* = \frac{\sum_{i=1}^{n}\left(g_i - I_g^n\right)\left(f_i - I_f^n\right)}{\sum_{i=1}^{n}\left(f_i - I_f^n\right)^2}$$

However this may introduce bias in the MC result:

$$E\left[\hat{\beta}^*\left(\frac{1}{n}\sum_{i=1}^{n} f_i - \mu_f\right)\right] \neq 0$$

In practice:

- out of sample estimation of β^*
- $\beta = sign(\rho_{gf})b$, $sign(\rho_{gf})$ can be guessed from intuition, $b > 0$, can be estimated from experiments on small samples

Example:

$E_t\left[S_T\right] = S_t e^{(r-q)(T-t)}$ → use S as control variate

- Other variance reduction methods:
  - Moment Matching
  - Stratified sampling
  - ….

Practical issues:

- Valuation of multi-asset derivatives
- Valuation of American style derivatives
- Reproducibility – initialize with seed
- Quality of random number generator
- Calculation of Greeks